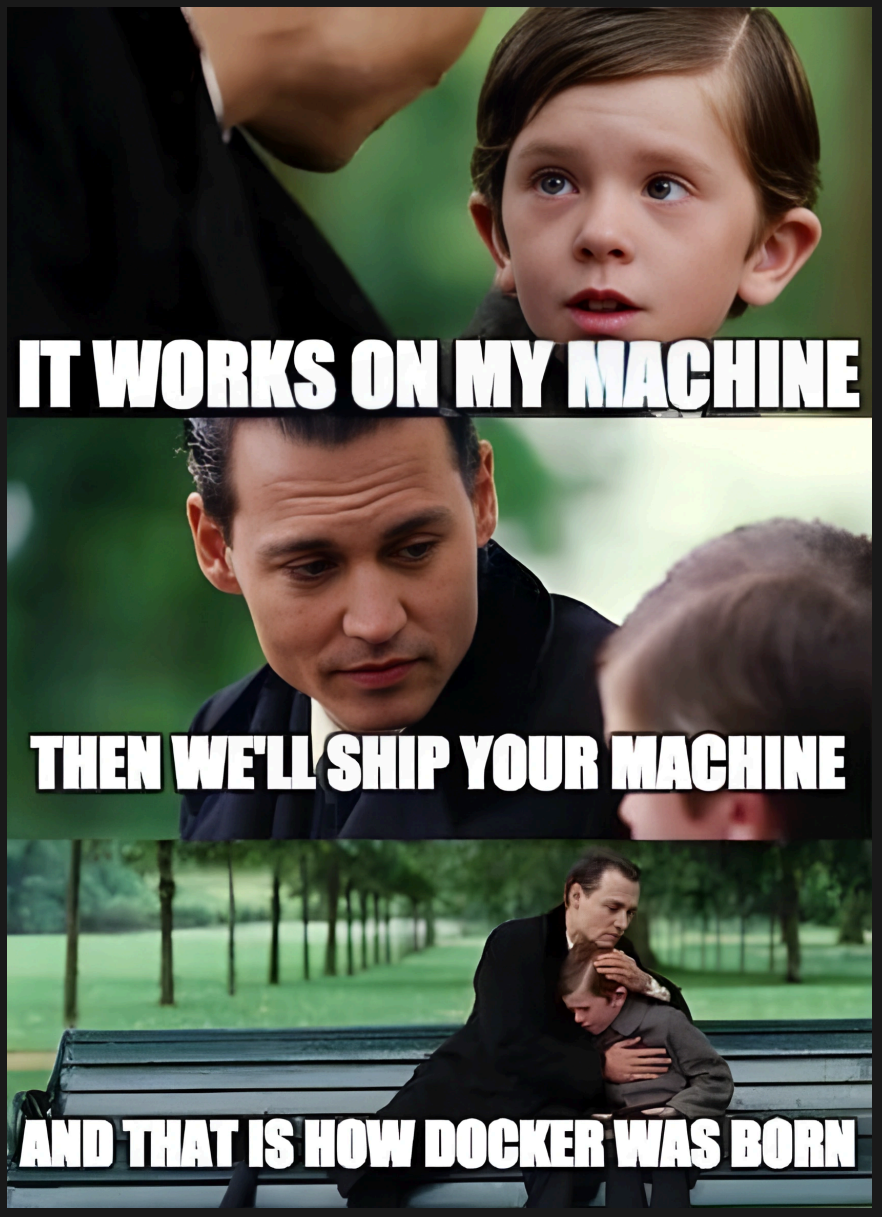
DHBW


IT WORKS ON MY MACHINE
THEN WE'LL SHIP YOUR MACHINE
AND THAT IS HOW DOCKER WAS BORN

DOCKER

How can Docker help solve the problem "It works on my machine"?

Out of scope

These topics will not be part of this presentation

- **Linux Basics**
- **Docker Desktop** I will do everything with the Bash commands
- **Docker Volumes**
- **Docker Networks**

WHY DO WE NEED DOCKER?

CRAFTY CONTROLLER: AUTOMATED INSTALL

Source: docs.craftycontrol.com - Linux installation

```
sudo apt update && sudo apt upgrade && sudo apt i
```

2. Run the installer

```
git clone https://gitlab.com/crafty-controller/cr
cd crafty-installer-4.0
sudo ./install_crafty.sh
```

3. Start Crafty manually (default dir: /var/opt/minecraft/crafty)

```
sudo su crafty
cd /var/opt/minecraft/crafty
 /run_crafty_sh
```

THE SAME WITH DOCKER

One command instead of installer, users and service files

```
docker run \
  --name crafty_container \
  --detach \
  --restart always \
  -p 8443:8443 \
  -p 8123:8123 \
  -p 19132:19132/udp \
  -p 25500-25600:25500-25600 \
  -e TZ=Etc/UTC \
  -v "/$(pwd)/docker/backups:/crafty/backups" \
  -v "/$(pwd)/docker/logs:/crafty/logs" \
  -v "/$(pwd)/docker/servers:/crafty/servers" \
  -v "/$(pwd)/docker/config:/crafty/app/config" \
```

BASICS

- What is a container?
- Container vs VM
- What is an image?
- What is a registry?
- Important Commands

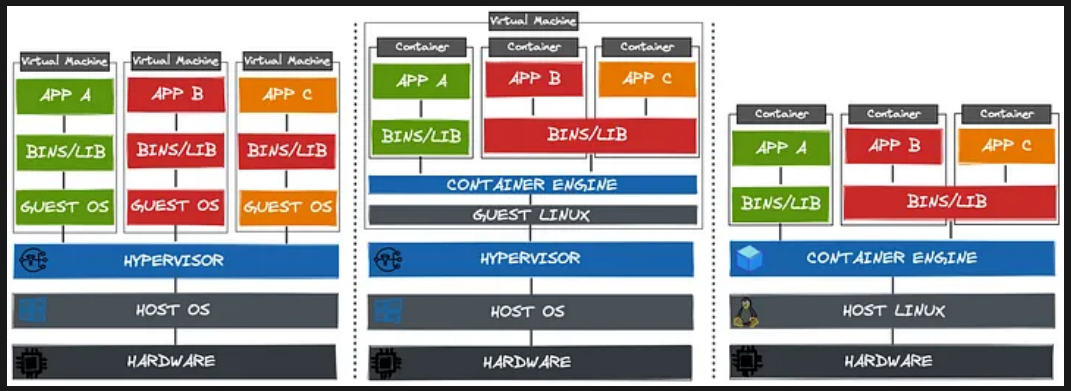
WHAT IS A CONTAINER?

- Isolated environment
- Install dependencies for every container
- Every container has its own namespace
- cgroups

Sources: How does Docker actually work?, namespaces(7), cgroups(7)

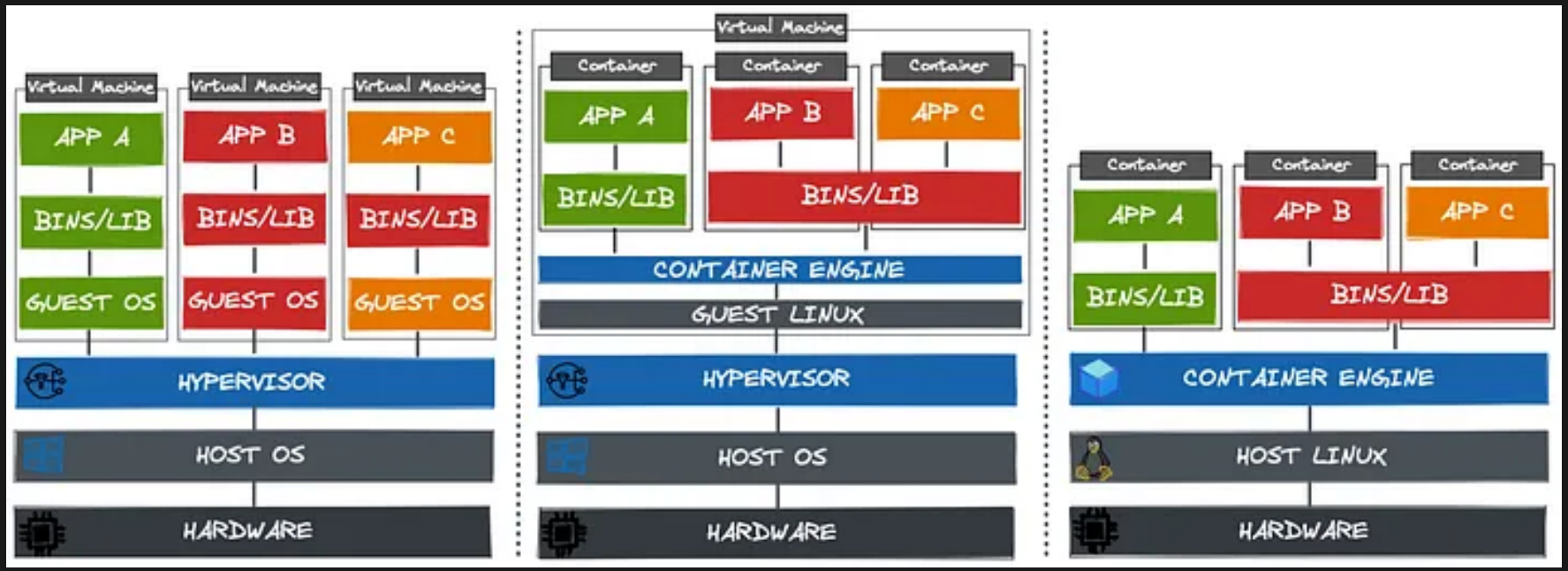
CONTAINER VS VM

- **VM:** full guest OS + kernel per app
- **Container:** shares the host kernel
- **Ubuntu 24.04 VM:** min. 2.5 GB
- **Ubuntu 24.04 container:** ~80 MB



Sources: How does Docker actually work?, Ubuntu Server system requirements, Docker lightweight images explained

CONTAINER VS VM



WHAT IS AN IMAGE?

- Read-only file
- Uses layers to store information
- Based on OCI Runtime Specification
- Used to create a container

Source: How does Docker actually work?

WHAT IS A REGISTRY?

Used to share images with other devices

Docker Hub is the standard registry Docker uses.

You can find a lot of standard images for Ubuntu and more

You can self-host your own

Source: Docker Docs: What is a registry?

IMPORTANT COMMANDS

`docker ps`	Lists all running containers
`docker image ls`	Lists all downloaded images
`docker run <image>`	Creates and starts a container
`docker <start|stop|rm> <container hash|name>`	Runs the given action on a container
`docker build -t <tag> <build_context>`	Builds an image from a Dockerfile
`docker logs <container hash|name>`	Prints the logs of a container

HOW DO WE CREATE AN IMAGE?

Dockerfile = the blueprint of an image

```
FROM node:26-alpine

WORKDIR /app

COPY . .
RUN npm install --omit=dev

EXPOSE 3500

CMD ["node", "app.js"]
```

Source: How does Docker actually work?

.DOCKERIGNORE

- Excludes files from the build context
- Ignored files do not end up in the image
- Works like a `.gitignore`

```
# .dockerignore (example)
node_modules
.git
.env
```

Source: How does Docker actually work?

MULTI-STAGE BUILDS

Source: Docker Docs: Multi-stage builds

MULTI-STAGE BUILDS

- Problem: build tools and dev dependencies bloat the image

Source: Docker Docs: Multi-stage builds

MULTI-STAGE BUILDS

- Problem: build tools and dev dependencies bloat the image
- Several `FROM` stages in one Dockerfile

Source: Docker Docs: Multi-stage builds

MULTI-STAGE BUILDS

- Problem: build tools and dev dependencies bloat the image
- Several `FROM` stages in one Dockerfile
- Build in one stage, copy only the result into the final image

Source: Docker Docs: Multi-stage builds

MULTI-STAGE BUILDS

- Problem: build tools and dev dependencies bloat the image
- Several `FROM` stages in one Dockerfile
- Build in one stage, copy only the result into the final image
- Result: smaller and safer images

Source: Docker Docs: Multi-stage builds

MULTI-STAGE EXAMPLE

```
# Stage 1: build
FROM node:26-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Stage 2: runtime image
FROM node:26-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY --from=build /app/dist ./dist
```

Source: Docker Docs: Multi-stage builds

DOCKER COMPOSE

Don't want the hassle of starting every container yourself?

Goal: frontend, backend and database on the same server

- The `compose.yml` file (older: `docker-compose.yaml`) defines the **containers** (services)
- Use the `docker compose` command in the folder of this file

Source: Docker Docs: Compose

COMPOSE.YML

```yaml
services:
  db:
    image: mariadb:11.4
    restart: unless-stopped
    environment:
      MARIADB_ROOT_PASSWORD: ${MARIADB_PASSWORD:?set MARIADB_PASSWORD in .env}
      MARIADB_DATABASE: ${MARIADB_DATABASE:-testdb}
    volumes:
      - db-data:/var/lib/mysql
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro

  app:
    build: ./backend
    restart: unless-stopped
    environment:
      MARIADB_HOST: db
      MARIADB_PORT: 3306
      MARIADB_USER: root
      MARIADB_PASSWORD: ${MARIADB_PASSWORD}
      MARIADB_DATABASE: ${MARIADB_DATABASE:-testdb}
```

DOCKER COMPOSE COMMANDS

`docker compose up`	Starts all containers (services)
`docker compose stop`	Stops all containers (services)
`docker compose down`	Stops and deletes all containers (services)
`docker compose logs`	Prints the logs of all containers (services)

SOURCES

- How does Docker actually work?
- Docker lightweight images explained
- Ubuntu Server system requirements
- namespaces(7)
- cgroups(7)
- Docker Docs: What is a registry?
- Docker Docs: Multi-stage builds
- Docker Docs: Compose